

Exercices

Exercice 1

sur un serveur linux ubuntu 24.04, configure openSSH pour permettre l'accès à distance et installer Docker.

suivre les consigne d'installation de portainer un gestionnaire de conteneurs web:

<https://docs.portainer.io/start/install/server/docker/linux>

répondre au question suivante:

- qu'est ce que le -d dans la commande docker run?
- quel sont les ports d'écoute de portainer?
- Est ce que les données de portainer sont persistantes après un redémarrage du conteneur?
- lister autre option pour le parametre --restart?

Exercice 2

Sur le même serveur, créer un dockerfile pour un script python qui enregistre la météo depuis open-meteo api à un interval de 10 secondes et affiche les données dans la console.

Ensuite chercher les messages de log du conteneur et vérifier que les données sont affichées.

vous pouvez travailler avec vscode en ssh vers le serveur pour créer les fichiers nécessaires.

<https://code.visualstudio.com/docs/remote/ssh>

- qu'elle image utiliser pour le dockerfile?
- qu'elle instruction permet de transferer le script python dans l'image?
- qu'elle instruction permet de lancer le script python à l'exécution du conteneur?

Exercice 3 compose

Sur votre serveur, utilisez Docker Compose pour déployer un serveur web Nginx qui diffuse une page HTML personnalisée.

- Créer un répertoire nommé site_web.
- À l'intérieur, créer un fichier index.html avec un message de bienvenue.
- Créer un fichier docker-compose.yml pour lancer un conteneur Nginx.
- Mapper le port 8080 de l'hôte vers le port 80 du conteneur.
- Utiliser un volume pour lier votre fichier index.html local au dossier /usr/share/nginx/html/ du conteneur.

tester la mise a jour de la page en modifiant le fichier index.html et en rafraichissant la page web.

Exercice 4 - Docker Compose multi-conteneurs (Nginx + API Python + Redis)

Déployer une petite application composée de plusieurs services qui communiquent entre eux avec Docker Compose.

Mettre en place :

- Une **API Python (Flask)** qui expose une route `/visites`
 - Un service **Redis** pour compter le nombre de visites
 - Un service **Nginx** qui agit comme reverse proxy vers l'API
 - L'ensemble orchestré avec **Docker Compose**
-

Étape 1 — Créer l'arborescence du projet

Créer la structure suivante :

```
app_compose/
├─ docker-compose.yml
├─ api/
│   └─ Dockerfile
│       └─ app.py
└─ nginx/
    └─ default.conf
```

Étape 2 — Écrire le script Python `api/app.py`

Code flask requis pour l'API Python qui incrémente un compteur de visites dans Redis et retourne le total.:

```
from flask import Flask
import redis
import os

app = Flask(__name__)

redis_host = os.getenv("REDIS_HOST", "redis")
r = redis.Redis(host=redis_host, port=6379, decode_responses=True)

@app.route("/visites")
def visites():
    total = r.incr("compteur")
    return f"Nombre de visites : {total}\n"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Étape 3 — Créer le `api/Dockerfile`

Contraintes :

- Partir d'une image Python légère
 - Définir un répertoire de travail
 - Copier le script Python dans l'image
 - Installer les dépendances `flask` et `redis`
 - Lancer automatiquement le script au démarrage du conteneur
-

Étape 4 — Créer la configuration Nginx `nginx/default.conf`

```
server {  
    listen 80;  
  
    location / {  
        proxy_pass http://api:5000/visites;  
    }  
}
```

Étape 5 — Écrire le `docker-compose.yml`

Le fichier doit définir trois services.

Service `redis`

- Utiliser l'image officielle Redis
- Politique de redémarrage automatique

Service `api`

- Construire l'image à partir du dossier `api`
- Définir la variable d'environnement `REDIS_HOST`
- Dépendre du service `redis`
- Politique de redémarrage automatique

Service `nginx`

- Utiliser l'image officielle Nginx
 - Mapper le port `8080` de l'hôte vers le port `80` du conteneur
 - Monter le fichier `default.conf` comme volume dans Nginx
 - Dépendre du service `api`
-

Étape 6 — Lancer l'environnement

Construire et démarrer les services avec Docker Compose.

Accéder à l'application via : `localhost:8080`

Rafraîchir plusieurs fois la page.

Étape 7 — Vérifications à effectuer

- Consulter les logs du service `api`
 - Redémarrer uniquement le conteneur `api` et observer le comportement
 - Redémarrer uniquement le conteneur `redis` et observer le comportement
 - Supprimer un conteneur et le recréer
 - Observer l'effet sur le compteur
-

Questions à répondre

- Pourquoi les services peuvent communiquer avec leur nom (`api`, `redis`) ?
- À quoi sert `depends_on` ici ?
- Où se trouve le réseau créé par Docker Compose ?
- Comment rendre les données Redis persistantes même après suppression du conteneur ?
- Comment démarrer 3 instances du service `api` avec Compose ?

Remise

Sur Léa, remettre seulement le fichier `docker-compose.yml` du l'exercice 4.